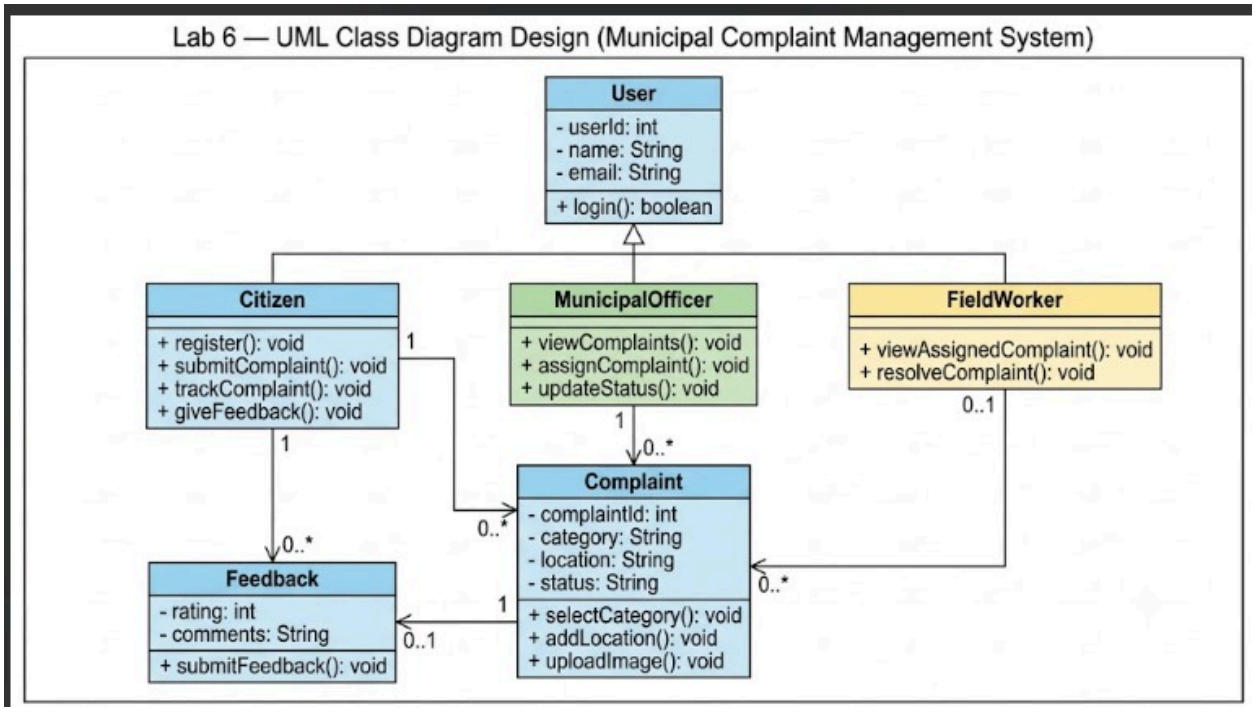
SECTION 1: LAB 6



— UML USE CASE MODELING

1.1 Introduction

Use case modeling is used to capture the functional requirements of the system by illustrating the interaction between external actors and the system boundaries. It defines *what* the system does from a user's perspective rather than *how* it does it.

1.2 System Boundary & Scope

- **System Boundary:** "Municipal Complaint Management System" — encapsulates all portal features, role verification databases, and status routing modules.
- **Core Scope:** End-to-end management of civic complaints, starting from citizen registration and grievance logging to administrative review, field dispatch, status updates, and feedback submission.

1.3 System Actors

1. **Citizen:** The primary end-user who logs grievances, monitors tracking states, and provides satisfaction reviews.
2. **Municipal Officer:** The administrative controller who inspects incoming cases, manages department workflows, and assigns tickets to ground workers.
3. **Field Worker:** The operations staff member who views assigned tasks and performs on-site remediation.

1.4 Detailed Use Cases & Relationships (<<include>> / <<extend>>)

- **Register / Login:** Authenticates citizens, officers, and workers into their respective dashboards.
- **Submit Complaint:** Core citizen action, which mandatorily includes:
    - Select Category: Choosing the department/issue type.
    - Add Location: Capturing GPS or manual street addresses.
    - Upload Image: Attaching photo evidence.
- **Track Complaint:** Allows citizens to invoke View Complaint Status to check real-time database flags.
- **Give Feedback:** An extension (<<extend>>) of tracking, activated once a complaint is resolved, allowing ratings and comments.
- **Assign Complaint & Update Status:** Performed by municipal officers to distribute tasks and transition lifecycle states.
- **Resolve Complaint:** Executed by field workers after accessing View Assigned Complaint.

SECTION 2: LAB 6 — UML CLASS DIAGRAM DESIGN

2.1 Introduction

The UML Class Diagram outlines the static object-oriented architecture, defining data structures, attributes, method signatures, visibility flags, and inter-class relationships (inheritance, association, composition).

2.2 Complete Class Specifications

- **User (Abstract Base Class)**
    - *Attributes:* - userId: String, - name: String, - email: String, - password: String
    - *Methods:* + login(): boolean, + logout(): void
- **Citizen (Derived Class extending User)**
    - *Attributes:* - citizenId: String, - address: String, - phone: String
    - *Methods:* + register(): void, + submitComplaint(): void, + trackComplaint(): void, + giveFeedback(): void
- **MunicipalOfficer (Derived Class extending User)**
    - *Attributes:* - officerId: String, - department: String

    - *Methods:* + viewComplaints(): void, + assignComplaint(): void, + updateStatus(): void

- **FieldWorker (Derived Class extending User)**
    - *Attributes:* - workerId: String, - zone: String, - availability: boolean
    - *Methods:* + viewAssignedComplaint(): void, + resolveComplaint(): void
- **Complaint (Core Entity Class)**
    - *Attributes:* - complaintId: String, - category: String, - location: String, - imageURL: String, - status: String, - createdDate: Date
    - *Methods:* + selectCategory(): void, + addLocation(): void, + uploadImage(): void, + getStatus(): String
- **Feedback (Entity Class)**
    - *Attributes:* - feedbackId: String, - rating: int, - comments: String
    - *Methods:* + submitFeedback(): void

2.3 Relationships & Cardinality Rules

- **Generalization:** Citizen, MunicipalOfficer, and FieldWorker inherit from User via white-triangle inheritance links.
- **Citizen to Complaint (1 to 0..*):** One citizen can lodge multiple complaints over time.
- **Municipal Officer to Complaint (1 to 0..*):** One officer oversees and routes multiple complaints.
- **Field Worker to Complaint (0..1 to 0..*):** Complaints are allocated to field workers for resolution.
- **Complaint to Feedback (1 to 0..1):** A resolved complaint links to at most one feedback instance.